

Simple Music Genre Classification

Sam Edwards

Introduction: Significance

Personalization

- Helps services make recommendations based on genre.

Playlist Creation

- Allows for automatic generation of a playlist.

Discovery

- Exposes users to new music.

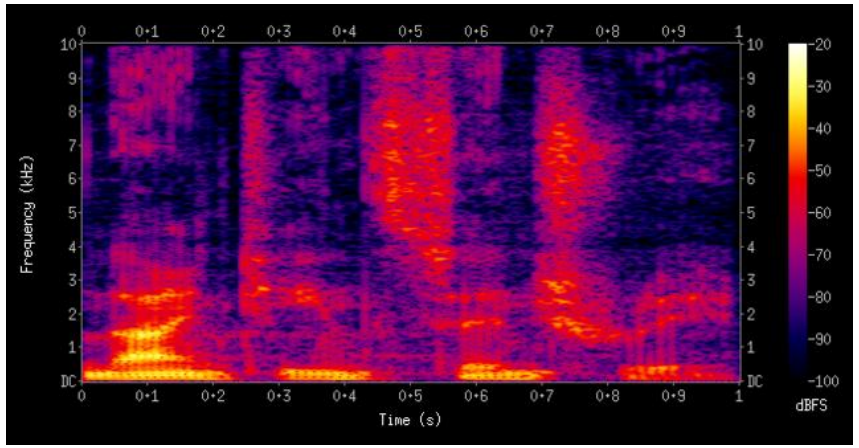
Introduction: Project Scope & Complex vs. Simple Classification

Complex Classification

- **500**+ Genres
- **1**+ label(s) per song
- Rhythm & Tempo
- Instrumentation & Harmony
- Spectral Features

Simple Classification

- **4-5** Genres
- **1** label per song
- Spectral Features

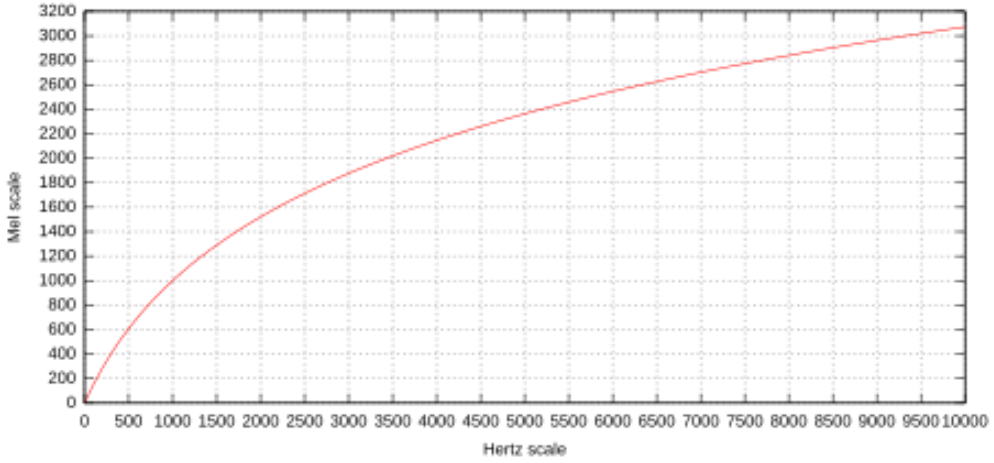


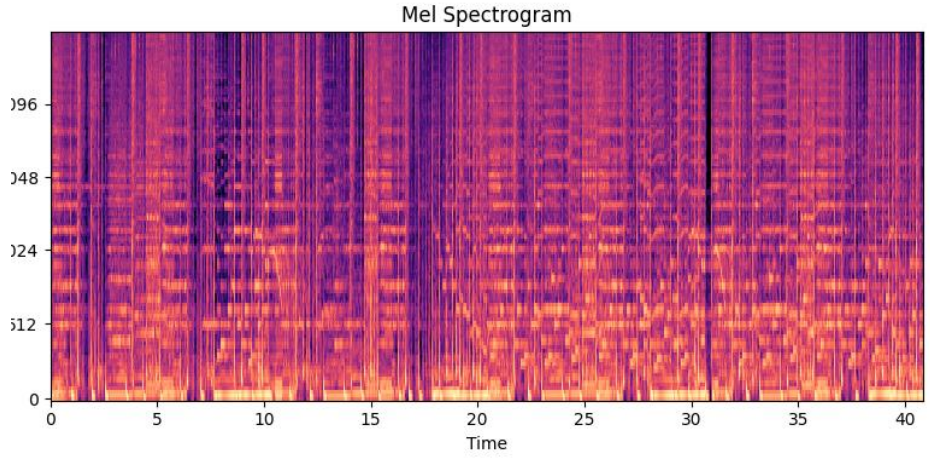
Background: Genre Classification

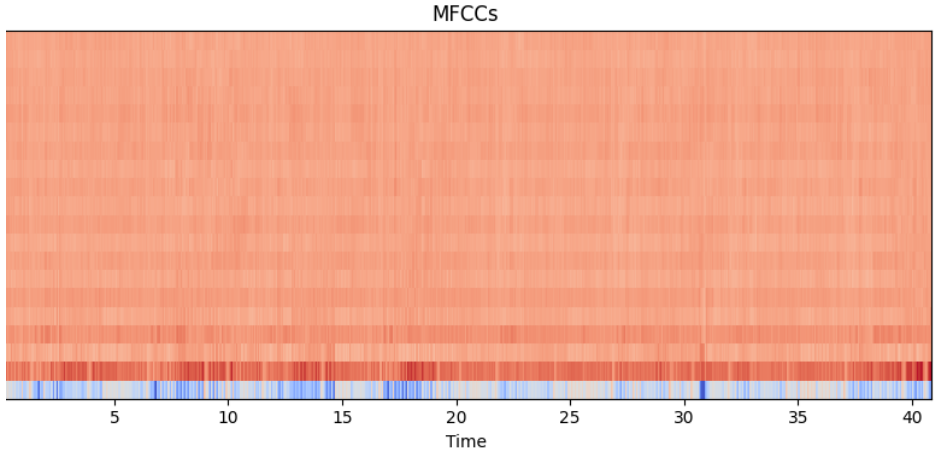
- Music Genre Classification
  - The task of automatically categorizing music tracks into predefined genre categories (e.g., electronic, rock, folk, classical) using machine learning algorithms.
- Used models/algorithms in the past
  - K-NN, SVM, Random Forests, CNNs, NNs, RNNs, LSTMs, & more…
- Techniques used in the past
  - Fourier Transform -> Spectrograms, frequency features
  - Spectral Features -> Mel-Frequency Cepstral Coefficients (MFCCs)
  - Chroma Features -> Harmonic, melodic features
  - Rhythmic Features -> Tempo, beat, groove features

Background: Spectral Features

- Mel-Frequency Cepstral Coefficients (MFCCs)
  - Short term power spectrum of an audio signal.
  - Allows for the capture of “timbral texture” or tone.
  - Represented similar to how human auditory perception could be graphed.
- Creation Process (Audio to MFCC)
  - Convert audio from time to frequency.
  - Apply Fourier Transform to obtain a spectrogram
  - Map frequencies to the Mel Scale
  - Apply logs of the power at each Mel frequency
  - Take the discrete cosign transform of the list of Mel log powers.
  - The MFCCs are the amplitudes of the resulting spectrum.

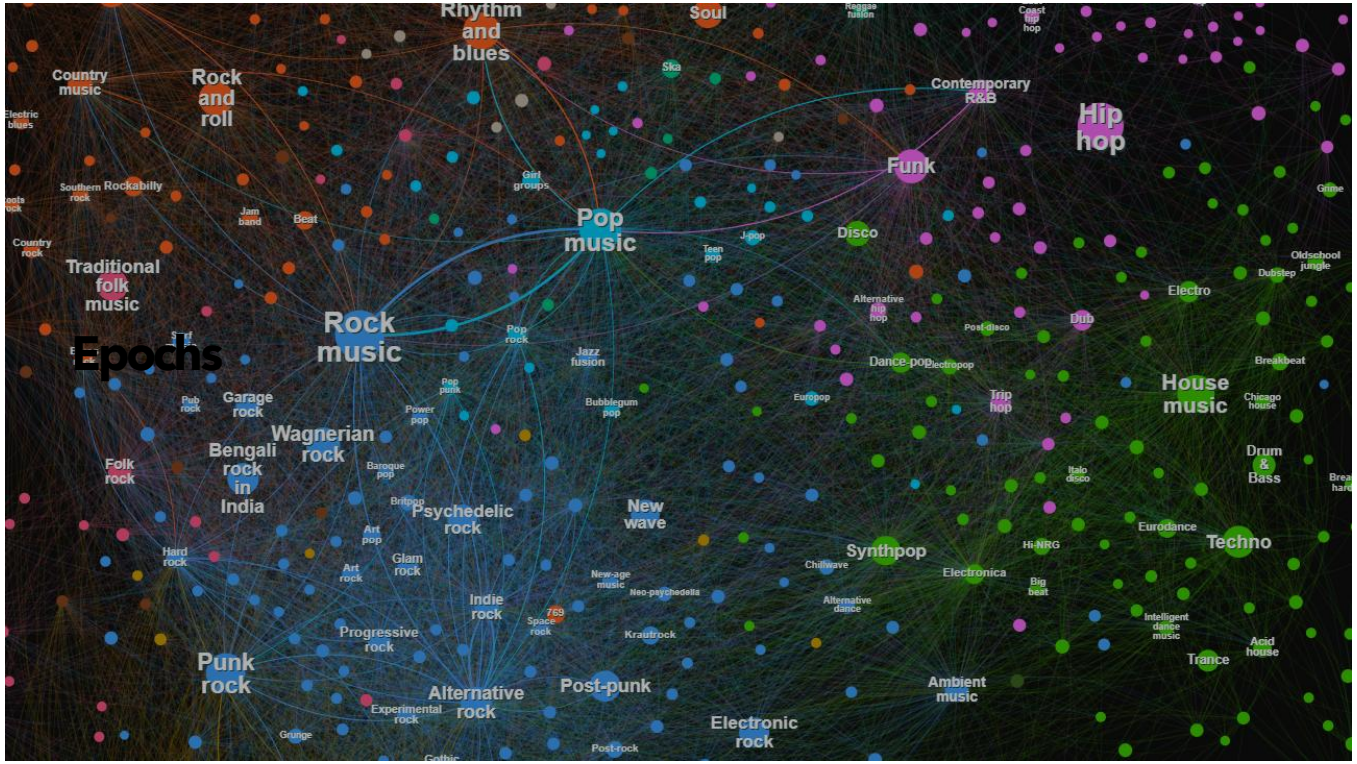






The Problem with Music Classification

There are many unique genres, and even more that sound similar to others.



Approach: Simple Classification

Genres Selected

- Electronic (2999 samples)
- Rock (2975 samples)
- Folk (1308 samples) *(Mix of Folk & Country)**
- Classical (479 samples)

General Classification

- Each sample has one of these four genres as its label.
- After classification, samples will be given a percentage of each genre, totaling 100%.
- The highest percentage is the classified genre for the sample.

Approach: Data Extraction & Cleaning

- Data Extraction
  - **20** Mel-frequences Cepstral Coefficients (MFCCs) per audio sample
  - Means & Variations
  - Uses significantly less resources when extracting or training a model. (3 hours vs. 32 hours.)

Turn bytes into MFCC Means and variations


```
{'bytes':
b'OggS\x00\x02\x00\x00\x00\x00\x00\x00\x00\x00\xfe\xf4\x8fb\x00\x00\x00\x00T\xec\\\xc2\x01\x1e\x01
libvorbis\x01\x05vorbis%BCV\x01\x00@\x00\x00$s\x18*F\xa5s\x16\x84\x10\x1aBP\x19\xe3\x1cB\xcek\xec\
\x84\x0fB\xb8\x9a\x83\xac9\x08!\x84$5HP\x83\x069\xe8\x1c\x84\xc2,
```


mfcc1_mean	mfcc1_var	mfcc2_mean	mfcc2_var	mfcc3_mean	mfcc3_var	mfcc4_mean	mfcc4_var	mfcc5_mean	...
-12.125682	3579.614258	107.077087	381.908112	-29.178938	211.675659	53.150368	161.219772	-19.724335	...

This is ~1/10000** of ONE datapoint Dataset: ~10 GBs*

- Cleaning
  - Removed Unwanted Genres (Such as “Ambient Electronic”, “Experimental”, “International”)
  - Grouped Similar Genres Together (Folk & Country)
- Dataset: **19900** -> **7761** used

Approach: Models

Random Forest Classifier

- Simple, but effective
- Works well with MFCCs and smaller datasets
- Less sensitive to noise

Convolutional Neural Network

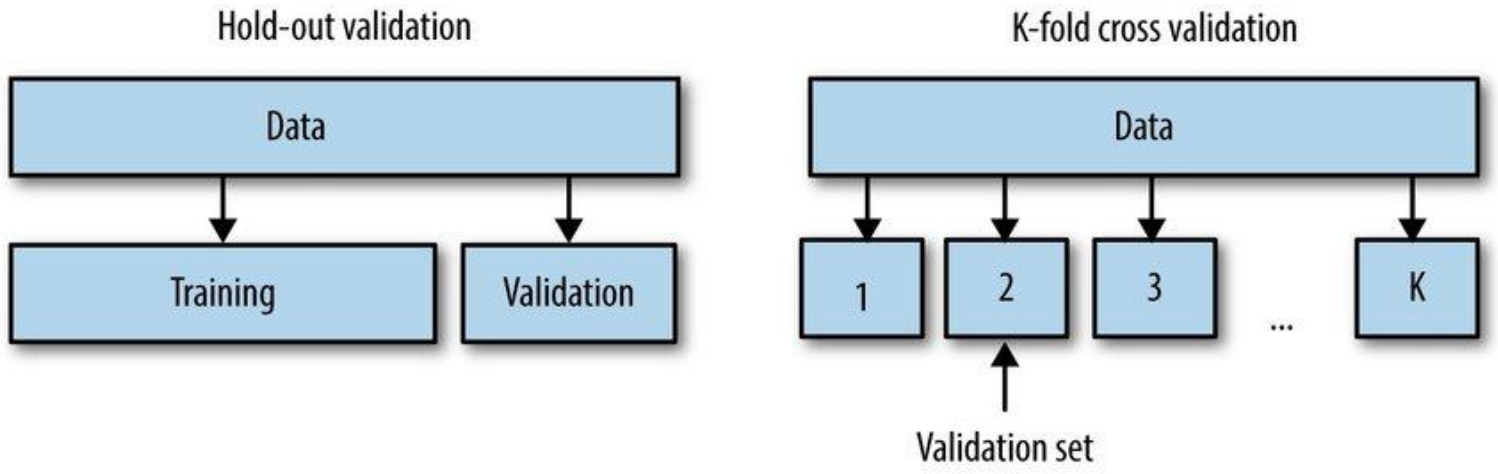
- Able to analyze MFCCs as an image
- Good at pattern recognition (Frequencies)

LSTM Recurrent Neural Network

- Good for sequential data
- Works well with audio segments
- Less sensitive to noise

Approach: Data Splitting

- Method 1: Hold-out Validation
  - **70 /15 /15**
  - Train/Validation/Test
- Method 2: Stratified K-Fold Validation (Folds = 5)
  - **80 (80 /20) /20**
  - Train(Train/Validation)/Test



Approach: Validation & Overfitting Avoidance

- Hold-out Validation
  - CNN Model (Originally used on LSTM too)
  - Simple, works well with large dataset
  - Data Dependency
- Stratified K-Fold Validation
  - LSTM Model
  - Complex, Splits dataset equally on folds.
  - Accounts for unbalanced datset.
  - Increased Computation = Reliable Model Evaluation
- Early Stopping
  - Monitored validation loss

```
#Define Early Stopping
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)
```

```
space = [
      Real(1e-5, 1e-2, "log-uniform", name="learning_rate"),
      Integer(64, 256, name="num_units_lstm1"),
      Integer(32, 128, name="num_units_lstm2"),
      Categorical([32, 64, 128], name="batch_size"),
  ]
```

```
result = gp_minimize(
    objective_function,
    space,
    n_calls=10,  # Number of iterations
    random_state=42,
)
```

Approach: Hyperparameter Tuning

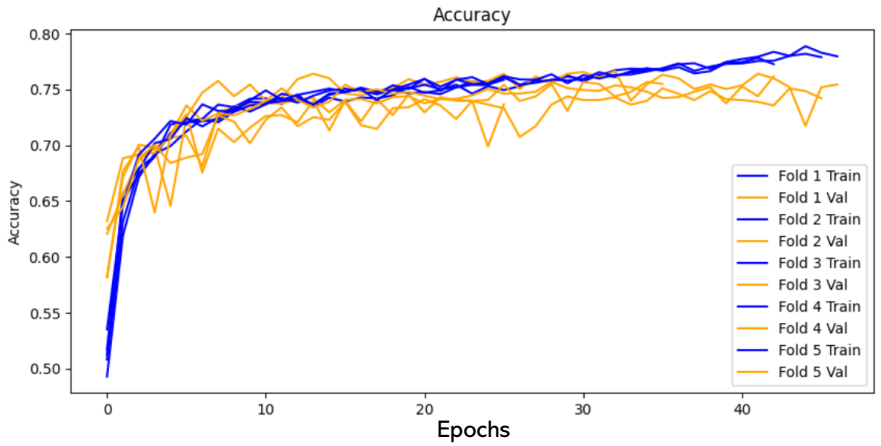
- Bayesian optimization using Gaussian Processes.
- Tuned Hyperparameters for Random Forest
  - n_estimators: **656**
  - max_depth: **13**
  - criterion: entropy

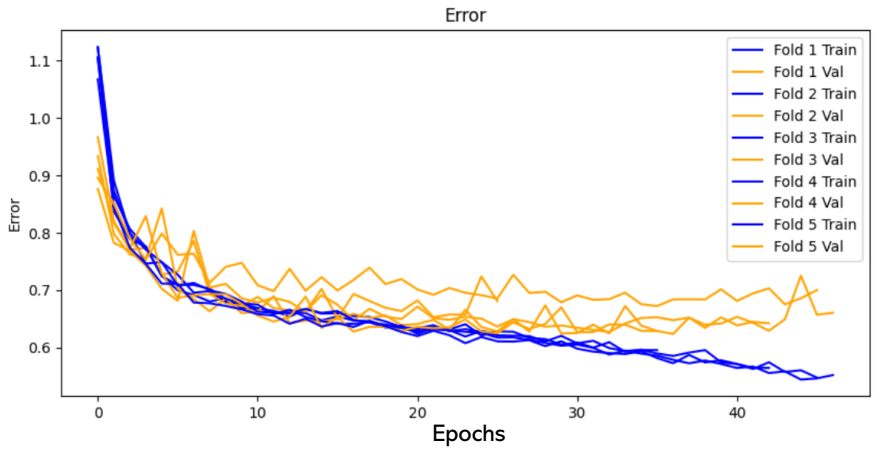
- Tuned Hyperparameters for LSTM Model
  - Learning Rate: **0.00245261263113367 9**
  - LSTM Layer 1: **99**
  - LSTM Layer 2: **107**
  - Batch Size: **64**

Approach: Initial Test Results

- Accuracy on Test dataset (15%)
  - Random Forest: ≈ **74.8**%
  - CNN: ≈ **73.2**%
  - LSTM ≈ **74.5**% *(Hold out Validation)*
- Accuracy on Test Dataset (20%)
  - LSTM: ≈ **75.5**% *(Stratified K-Fold)*

- Loss on Test dataset (15%)
  - Random Forest: ≈ **0.6747**
  - CNN: ≈ **0.6918**
  - LSTM: ≈ **0.6650** *(Hold out Validation)*
- Loss on Test Dataset (20%)
  - LSTM: ≈ **0.6300** *(Stratified K-Fold)*

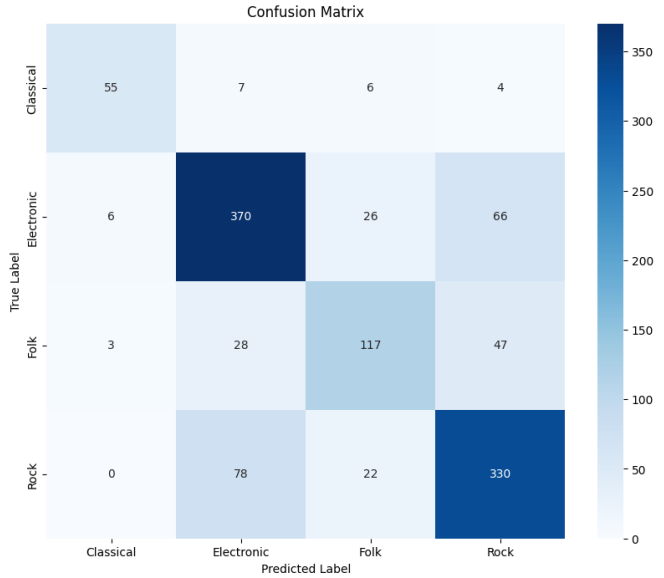




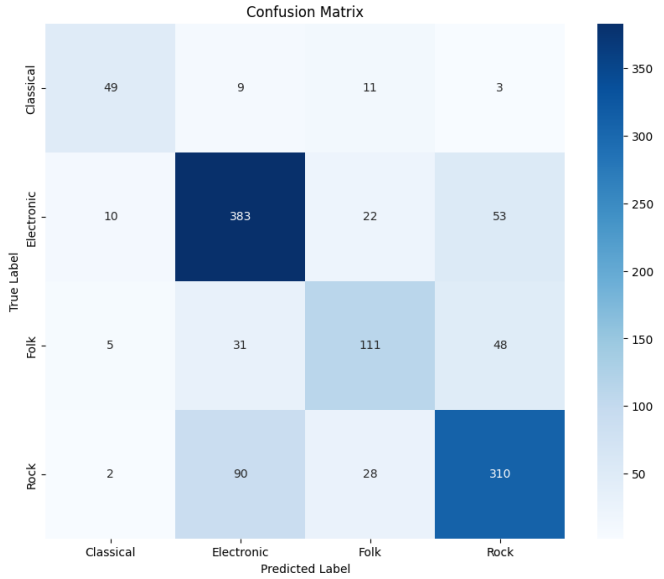
**Graphs visualized training data*

Results: Model Comparisons

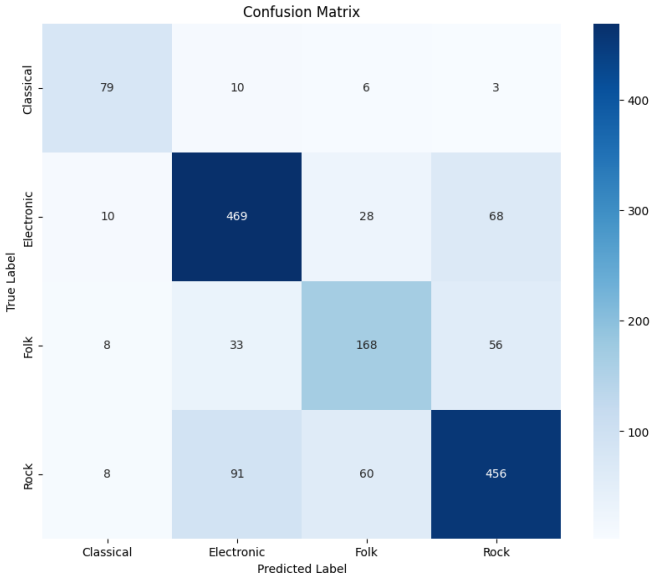
- Random Forest (15%)
  - Precision: ≈ **0.7479**
  - Recall: ≈ **0.7485**
  - F1-score: ≈ **0.7474**



- CNN (15%)
  - Precision: ≈ **0.7301**
  - Recall: ≈ **0.7321**
  - F1-score: ≈ **0.7299**



- LSTM (20%)
  - Precision: ≈ **0.7546**
  - Recall: ≈ **0.7547**
  - F1-score: ≈ **0.7542**



Results: Model Comparisons Cont.

- Random Forest (15%)
  - Standard Deviation of CV Scores: ≈ **0.0085**
  - 5 folds, Entire Dataset
  - Accuracy on Specific Labels

Label	Accuracy
Folk	0.600000
Classical	0.763889
Rock	0.767442
Electronic	0.790598

- CNN* (15%)
  - Standard Deviation of CV Scores: ≈ **0.0331**
  - 5 folds, 85% of dataset

**The CNN did NOT like the smaller split datasets*
Average Accuracy: ***59.9%***

  - Accuracy on Specific Labels

Label	Accuracy
Folk	0.569231
Classical	0.680556
Rock	0.720930
Electronic	0.818376

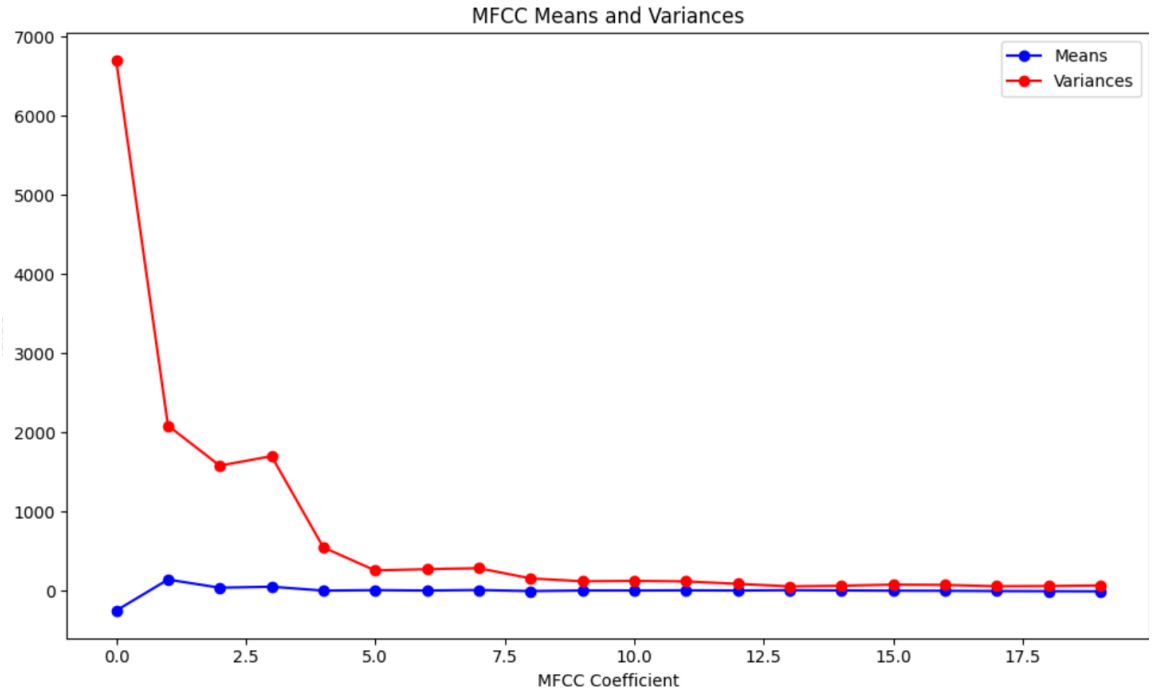
- LSTM (20%)
  - Standard Deviation of CV Scores: ≈ **0.0066**
  - 5 Folds, 80% of Dataset
  - Accuracy on Specific Labels

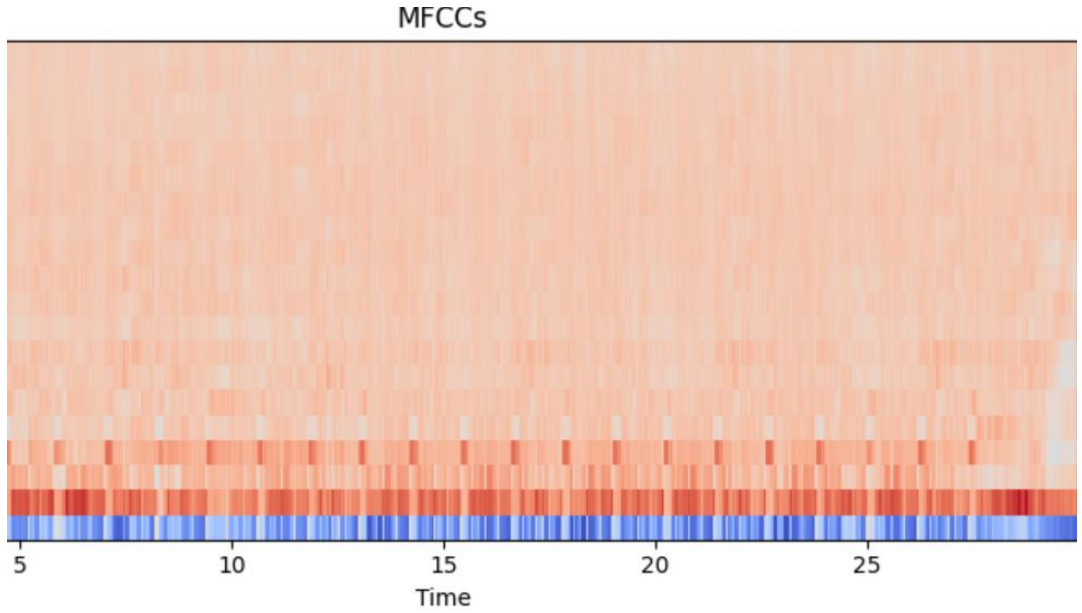
Label	Accuracy
Folk	0.633962
Classical	0.806122
Rock	0.741463
Electronic	0.815652

*CNN Ran with K-Fold Validation to get Standard Deviation of Cross Validation Scores

Discussion: Issues and Reasoning

- Bad Data.
  - Misleading labels, garbage data. Probably make up ≈10-20% of error.
- Dataset not big enough
  - Good datasets for Music genre classification are **500,000+** songs.
- Models
  - Random Forest Model: Performed well on smaller dataset.
  - CNN: MFCCs **!=** MFCC variation and means. Lacking data.
  - LSTM: Would continue to improve with more data.





Discussion: Future Work

- Try again with more data and genres. ☺

Sources

- California State University. (n.d.). *Music genre classification using machine learning*. ScholarWorks. Retrieved December 6, 2024, from https://scholarworks.calstate.edu/downloads/73666b68n
- Nilesh M. Patil, Dr. Milind U. Nemade. 2017. Music Genre Classification Using MFCC and KNN. Retrieved December 6, 2024, from https://d1wqtxts1xzle7.cloudfront.net/...
- Gessle, G., & Åkesson, S. (2019). A comparative analysis of CNN and LSTM for music genre classification (Dissertation). Retrieved from https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-260138